

“ Un código elegante no es aquel que tiene menos líneas, si no el que saca mayor provecho de ellas ”
— Oscar Blancarte (2016)





**“Sólo el tiempo demostrará
si nuestro software, al
enfrentarse a los fuertes
vientos del cambio, está
asentado sobre roca firme
o sobre arena”**

“En las fases iniciales del desarrollo de un proyecto de software, durante sus primeras versiones, las nuevas funcionalidades fluyen de manera natural.

A medida que los proyectos van avanzando y creciendo, comienzan a surgir una serie de problemas y situaciones que muestran que algo no va bien, son síntomas de que nuestros diseños se están degradando .





CARACTERÍSTICAS

Síntoma	Definición	Consecuencias
Rigidez	Dificultad de realizar cambios en el software, incluso en tareas sencillas.	- Las estimaciones son más abultadas. - Añadir funcionalidades nuevas cuesta mucho cuando antes era más rápido.
Fragilidad	Tendencia a que el software se rompa en múltiples sitios cada vez que se realiza un cambio en él, incluso sin tener relación el cambio con lo que se rompe.	- Subir una nueva versión y que se rompan partes que aparentemente no tienen que ver con lo modificado. - Aparición de mayor cantidad de bugs, aumentando el coste de esfuerzo y tiempo en corregirlos.
Inmovilidad	Incapacidad de reutilizar piezas de código por el gran acoplamiento que tiene, haciendo que este sea inamovible cuando la funcionalidad es prácticamente la misma que la deseada.	- Aumenta la complejidad del diseño y del código al introducir código duplicado. - Complica el entendimiento del código y genera equivocaciones.
Viscosidad	La tendencia de que, en el ámbito del software, realizar la solución buena requiere mayor esfuerzo comparado con la solución mala y rápida y, en el ámbito del entorno de desarrollo, éste es lento e ineficiente.	- Provoca el efecto bola de nieve: cuanto más tarde se corrija esa deuda técnica más difícil y costoso será resolverlo. - Pérdida de tiempo, estrés, confusión, pasotismo, acciones inseguras, etc.

- Cuando empezamos a desarrollar software, es común que cada quien utilice su propia lógica, conocimientos y experiencia para crear código. Y esto muchas veces resulta en desarrollos complejos que sólo su creador entiende.
- Pero, **¿es posible desarrollar un módulo que otro programador pueda aprovechar entender y mejorar?**
- La respuesta está en los patrones de diseño. Estos básicamente son modelos muestra que sirven como guía para que los programadores trabajen sobre ellos.



Patrones de Diseño

design patterns

Patrones de Diseño

- La idea de los patrones comenzó en la arquitectura con Christopher Alexander? Los libros que publicó describen la manera de construir arquitectura dinámica que fue la idea inspiracional para el uso de patrones en software.
- Algunos han llamado al trabajo de Alexander “una nueva forma de arquitectura”, por lo que no es sorprendente que, mientras Alexander creó patrones de diseño para ayudar a informar el mundo físico de la arquitectura y la construcción, los arquitectos digitales reconocieran rápidamente la aplicación de sus teorías en su propio trabajo.
- Como explica *A Pattern Language* : “Cada patrón describe un problema que ocurre una y otra vez en nuestro entorno y luego describe la esencia de la solución a ese problema, de manera que puedes usar esta solución un millón de veces, sin tener que hacerlo dos veces de la misma manera”.

un patrón de diseño es una forma formal de documentar soluciones a problemas de diseño comunes.

- Alexander creó su lenguaje para resaltar las relaciones entre patrones, donde el todo es mayor que la suma de sus partes.
- En el diseño digital actual, este principio se resume así: un logotipo puede ser el mejor jamás creado, los colores seleccionados una representación precisa de la marca y el diseño un atractivo panorama informativo, pero si las conexiones entre todos los elementos carecen de significado, nada lo tendrá.

¿Para qué sirven los patrones de diseño en programación?

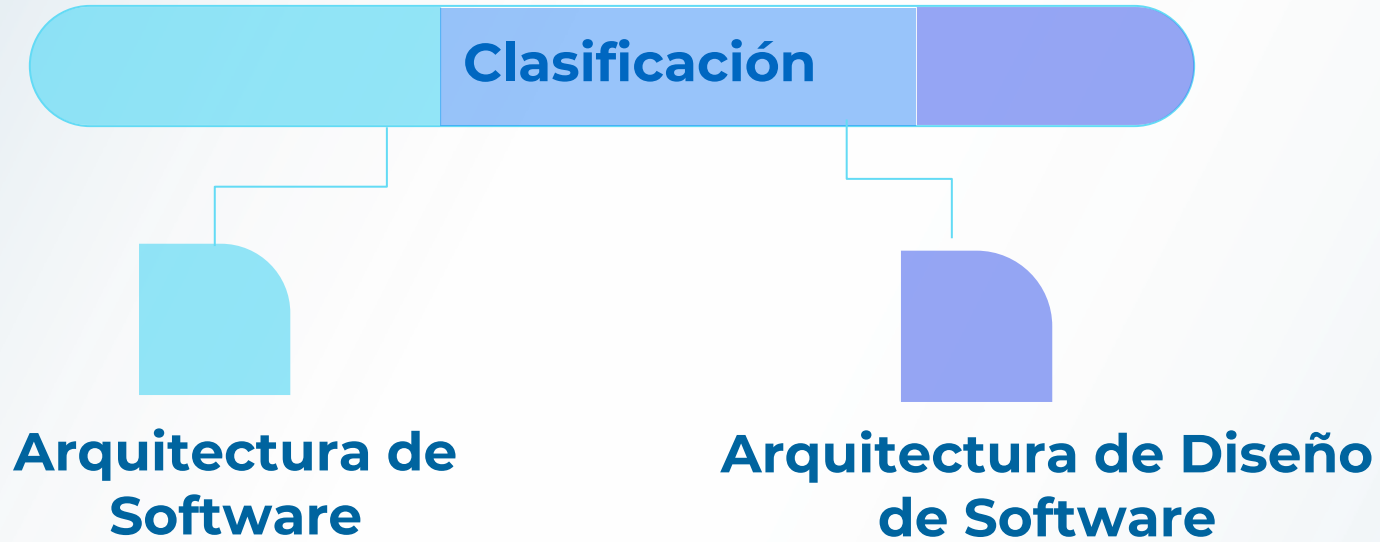
- Sirven para implementar una arquitectura y escribir código de la manera correcta y de este modo ayudar a resolver problemas de una manera estándar y profesional.

Los **PATRONES DE DISEÑO** o *DESIGN PATTERNS* son modelos muestra que sirven como guía para la búsqueda de soluciones a problemas comunes en el desarrollo de software y otros ámbitos del diseño de interacción o interfaces.

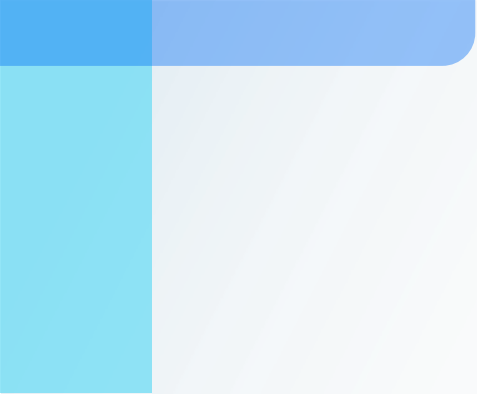
Los **PATRONES DE DISEÑO** o **DESIGN PATTERNS** son:

- una solución general, reutilizable y aplicable a diferentes problemas de diseño de software.
- son plantillas para soluciones a problemas comunes en el desarrollo de software que se pueden usar en diferentes contextos.
- Los patrones de diseño dan soluciones a problemas complejos sin importar el lenguaje que se esté usando.

Patrones de Diseño



- Puede parecer difícil distinguir entre patrones de arquitectura de software y patrones de diseño.
- Sin embargo, resulta más que transparente si tomamos ambos por separado. Los patrones de arquitectura se utilizan para crear la lógica de negocio, las interfaces de usuario y otros aspectos, mientras que el diseño de software se utiliza para implementar la lógica de negocio.



Patrones de Arquitectura de Software



Patrón de Arquitectura de Software

Un patrón arquitectónico es una representación pictórica de todo el sistema y sus subsistemas, así como de las funciones y responsabilidades, reglas e incluso una hoja de ruta para establecer las relaciones entre todas estas partes.

Patrón de Arquitectura de Software

- Un **patrón de arquitectura** representa la **relación entre los componentes** de todo el sistema y cada uno de ellos está **compuesto por pequeños módulos**.
- Afecta a la estructura global del sistema, ya que expresan un **esquema organizativo estructural para sistemas de software**, por lo tanto, **los patrones de diseño ayudan a comprender esa arquitectura, estructura o componente**.

Para la construcción de una aplicación se suele usar muchos patrones, por ejemplo:

Tipos de Arquitectura de Software

Monolítica

Arq. Orientada aServicios -SOA

Microservicios

Cliente-Servidor

Arq. en Capas

DevOps



Patrones de Diseño de Software



Los **PATRONES DE DISEÑO** o ***DESIGN PATTERNS***, son una solución general, reutilizable y aplicable a diferentes problemas de diseño de software.

La idea de los patrones de diseño fue popularizada por el libro *"Design Patterns: Elements of Reusable Object-Oriented Software"* (1994), escrito por Erich Gamma, Richard Helm, Ralph Johnson y John Vlissides, conocidos como el "Gang of Four" (GoF) . Este libro sentó las bases para entender cómo diseñar sistemas de software flexibles y robustos.

<https://www.javier8a.com/itc/bd1/articulo.pdf>

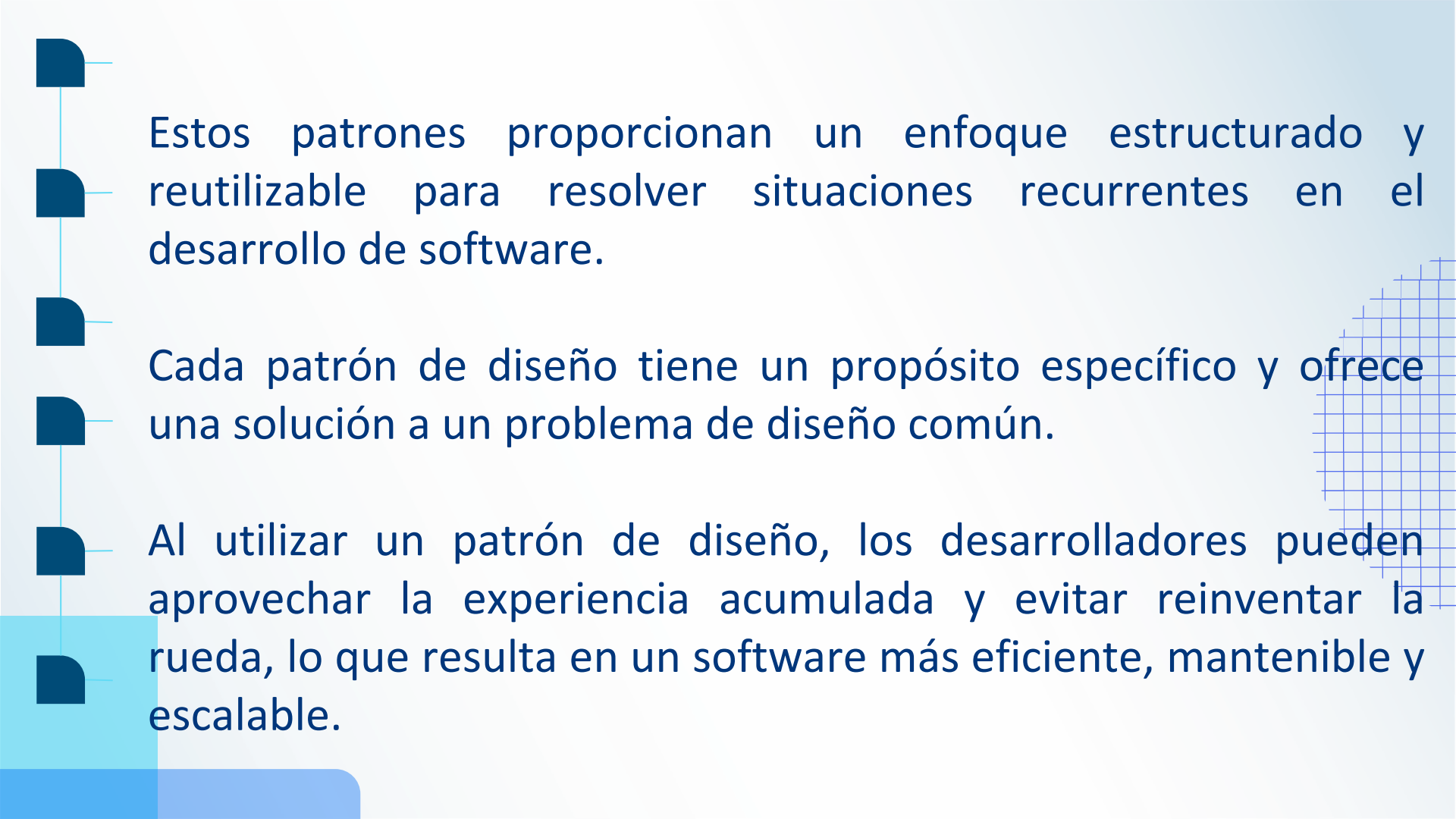
<https://profeuttec.yolasite.com/resources/Patrones%20de%20dise%C3%B1o%20-%20Erich%20Gamma.pdf>

PATRONES DE DISEÑO

Plantillas que identifican problemas en el sistema y proporcionan soluciones apropiadas a problemas generales a los que se han enfrentado los desarrolladores durante un largo periodo de tiempo, a través de prueba y error.

No son algoritmos ni código específico, sino más bien plantillas conceptuales que guían cómo estructurar y organizar el código para resolver problemas de manera eficiente, escalable y mantenible.





Estos patrones proporcionan un enfoque estructurado y reutilizable para resolver situaciones recurrentes en el desarrollo de software.

Cada patrón de diseño tiene un propósito específico y ofrece una solución a un problema de diseño común.

Al utilizar un patrón de diseño, los desarrolladores pueden aprovechar la experiencia acumulada y evitar reinventar la rueda, lo que resulta en un software más eficiente, mantenible y escalable.



Los patrones de diseño ayudan a los desarrolladores a comunicarse y compartir soluciones eficientes y efectivas que han demostrado ser exitosas en el pasado.



Es importante tener en cuenta que los patrones de diseño no son algoritmos o código listo para usar, sino más bien pautas y descripciones de soluciones de diseño. Los desarrolladores deben adaptar e implementar los patrones de diseño de acuerdo con los requisitos y características específicas de su proyecto de software.



Los patrones de diseño de software se basan en principios y prácticas de diseño que han evolucionado a lo largo del tiempo. A menudo, se describen utilizando terminología y notaciones específicas para facilitar su comprensión y aplicación.

Importancia de los patrones de diseño



1.Reusabilidad: Los patrones de diseño promueven la reutilización del diseño y del código, lo que ahorra tiempo y esfuerzo en el desarrollo de software.



2.Mantenibilidad: Al seguir patrones de diseño reconocidos, el código se vuelve más fácil de entender y mantener. Los desarrolladores pueden comprender rápidamente la estructura y el propósito del código, lo que facilita la corrección de errores y la incorporación de nuevas características.



Importancia de los patrones de diseño



- **Escalabilidad:** Los patrones de diseño proporcionan un marco flexible que permite que el software se adapte y crezca con los requisitos cambiantes del negocio. Esto facilita la escalabilidad del sistema a medida que se agregan más funcionalidades o se manejan volúmenes de datos más grandes.



- **Claridad y Consistencia:** Al seguir patrones de diseño establecidos, se promueve la consistencia en la estructura y el diseño del software. Esto facilita la colaboración entre desarrolladores y mejora la comunicación dentro del equipo de desarrollo.



Importancia de los patrones de diseño

- **Optimiza el Tiempo de Desarrollo** : En lugar de reinventar la rueda, los patrones ofrecen soluciones predefinidas que pueden implementarse rápidamente.
- **Previene Problemas Futuros** : Los patrones están diseñados para manejar situaciones complejas y cambios futuros, reduciendo la necesidad de refactorizaciones costosas.

Aunque los patrones de diseño son herramientas poderosas, deben usarse con moderación y criterio. Algunos puntos a tener en cuenta:

- **No forzar el uso de patrones** : Aplicar un patrón solo porque existe puede complicar innecesariamente el código.
- **Contexto es clave** : Un patrón que funciona bien en un escenario puede no ser adecuado en otro.
- **Simplicidad primero** : Si una solución simple resuelve el problema, no es necesario recurrir a un patrón complejo.



Tipos de Patrones de diseño



Creacional

1. Abstract Factory
2. Builder Patterns
3. Factory Method
4. Prototype
5. Singleton



Estructural

6. Bridge
7. Composite
8. Decorator
9. Facade
10. Flyweight
11. Proxy
12. Adapter



Comportamiento

13. Chain of responsibility
14. Command
15. Interpreter
16. Iterator
17. Mediator
18. Memento
19. Observer
20. State
21. Strategy
22. Template method
23. Visitor

Tipo de Patrón: CREACIONAL

Se encarga de la Inicialización y Configuración de objetos.

Esto le da al programa más flexibilidad para decidir qué objetos deben crearse para un caso de uso dado.

Tipo de Patrón: ESTRUCTURAL

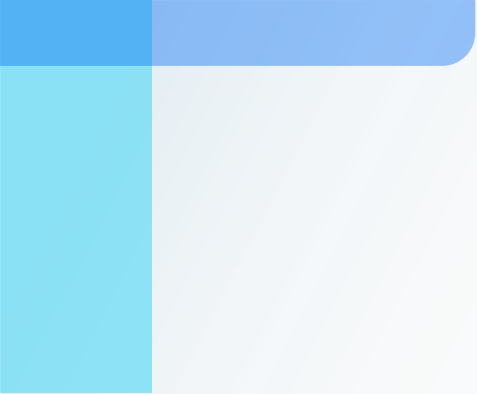
**Separan la interfaz de la implementación.
Se ocupan de cómo las clases y objetos se
agrupan para formar estructuras más
grandes.**

Tipo de Patrón: COMPORTAMIENTO

Se encarga en describir objetos o clases y la comunicación entre ellos.

Elementos de un patrón de diseño





conocimiento a la aplicación



A TRABAJAR!!!!!!!!!!!!!!